

Instituto Federal de Educação, Ciência e Tecnologia da Paraíba - IFPB

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Disciplina: Banco de Dados I - Período: 2025.2 - Professor: Hugo Feitosa de Figueirêdo

Equipe:

Geraldo Felix da Silva Junior

Pedro Henrique de Sousa Dantas Gomes

Gabriel Melo do Nascimento

Projeto de Banco de Dados

Sistema de Batalhas Pokémon

Minimundo

O sistema modela um ambiente onde treinadores podem montar times de Pokémon a partir de uma Pokédex e realizar batalhas entre si.

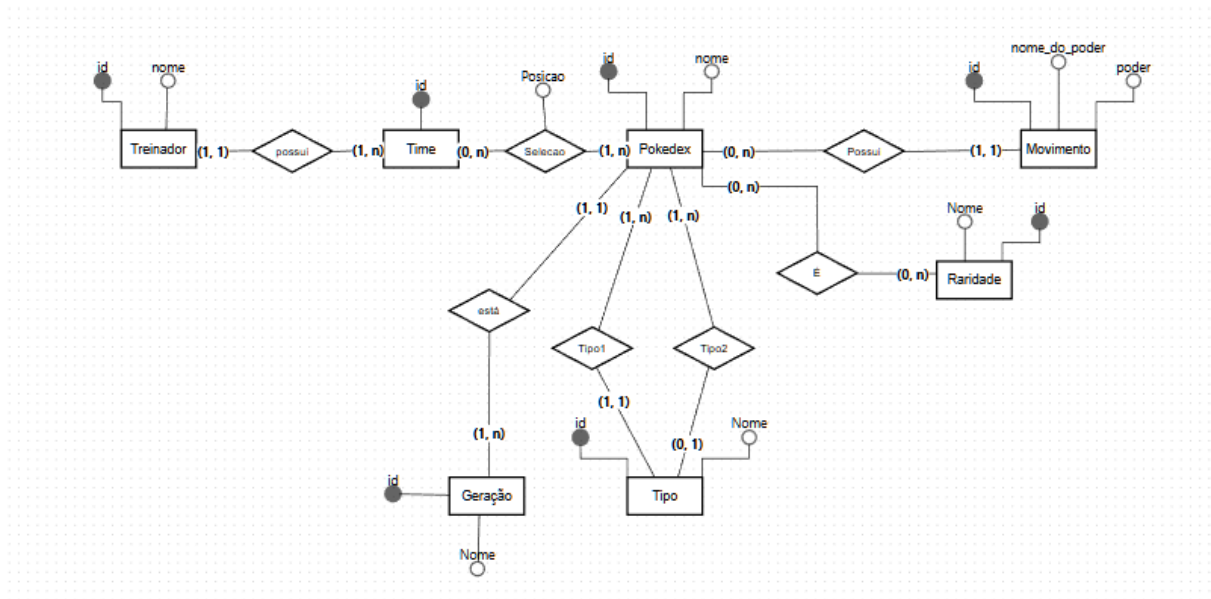
A Pokédex armazena informações dos Pokémon, incluindo nome, tipos, geração e um movimento principal, que possui um valor de poder associado.

Cada treinador pode possuir um ou mais times Pokémon. A composição de um time é feita por meio da seleção de até seis Pokémon distintos da Pokédex. Cada Pokémon ocupa uma posição única dentro do time, variando de 1 a 6.

As batalhas ocorrem entre dois times distintos, onde os Pokémon são confrontados de acordo com suas posições. O resultado de cada confronto é determinado pelo valor de poder do movimento do Pokémon. O Pokémon com maior poder vence a rodada; em caso de empate, nenhum dos dois vence a rodada.

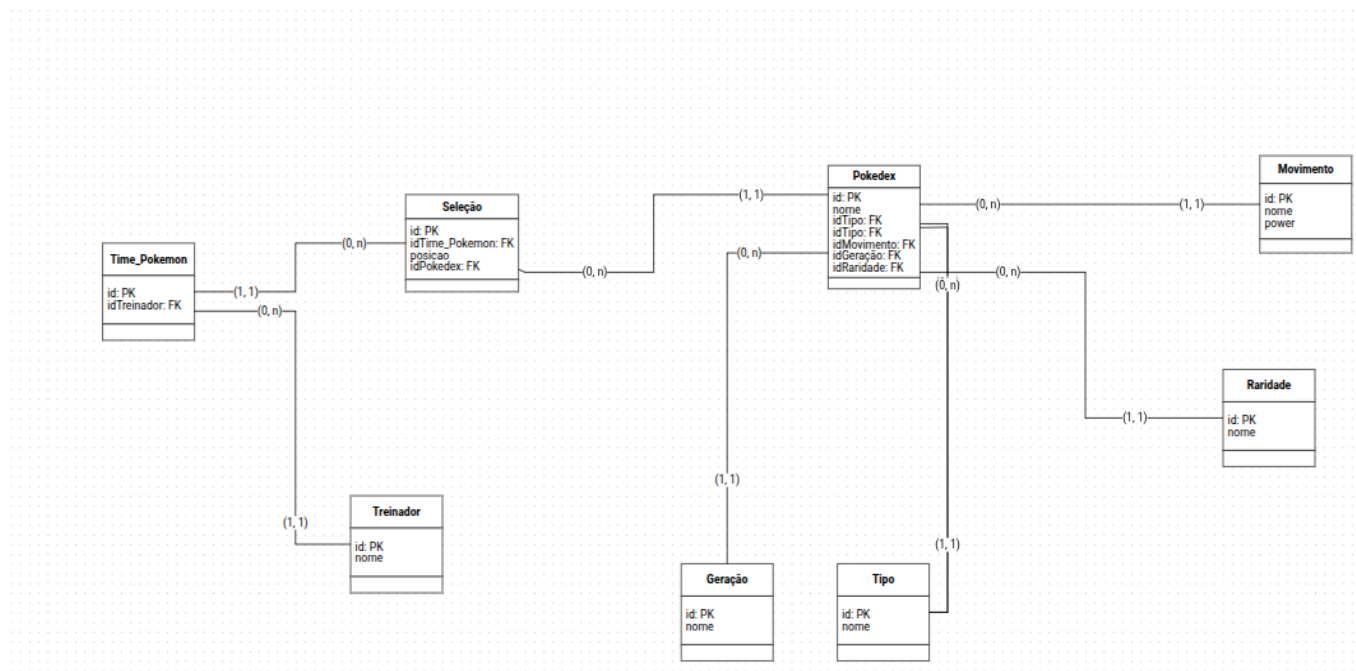
O treinador que obtiver o maior número de vitórias nas rodadas é declarado vencedor da batalha.

Esquema Conceitual



<https://app.brmodeloweb.com/#!/publicview/692a34ce0f216e436f4947e5>(LINK PÚBLICO)

Esquema Relacional



<https://app.brmodeloweb.com/#!/publicview/697807a9db85661c4be4b370> (LINK PÚBLICO)

Script SQL de criação das tabelas (CREATE TABLE)

```
CREATE TABLE IF NOT EXISTS Treinador (  
  
    id INTEGER PRIMARY KEY,  
  
    nome VARCHAR(255) NOT NULL  
  
);
```

```
CREATE TABLE IF NOT EXISTS Time_Pokemon (  
  
    id INTEGER PRIMARY KEY,  
  
    treinador_id INTEGER NOT NULL  
  
);
```

```
CREATE TABLE IF NOT EXISTS Time_Pokemon_Selecao (  
  
    id INTEGER PRIMARY KEY,
```

```
time_pokemon_id INTEGER NOT NULL,  
  
posicao INTEGER NOT NULL CHECK (posicao BETWEEN 1 AND 6),  
  
pokedex_id INTEGER NOT NULL,  
  
UNIQUE (time_pokemon_id, posicao)  
  
);
```

```
CREATE TABLE IF NOT EXISTS PokeDex (  
  
    id INTEGER PRIMARY KEY,  
  
    nome VARCHAR(255) NOT NULL,  
  
    tipo_id INTEGER NOT NULL,  
  
    tipo_2_id INTEGER NULL,  
  
    move_id INTEGER NOT NULL,  
  
    geracao_id INTEGER NOT NULL,  
  
    raridade_id INTEGER NOT NULL  
  
);
```

```
CREATE TABLE IF NOT EXISTS Tipo (  
  
    id INTEGER PRIMARY KEY,  
  
    nome VARCHAR(50) NOT NULL  
  
);
```

```
CREATE TABLE IF NOT EXISTS Movimento (
```

```
    id INTEGER PRIMARY KEY,
```

```
    nome VARCHAR(75) NOT NULL,
```

```
    power INTEGER NOT NULL
```

```
);
```

```
CREATE TABLE IF NOT EXISTS Geracao (
```

```
    id INTEGER PRIMARY KEY,
```

```
    nome VARCHAR(50) NOT NULL
```

```
);
```

```
CREATE TABLE IF NOT EXISTS Raridade (
```

```
    id INTEGER PRIMARY KEY,
```

```
    nome VARCHAR(50) NOT NULL
```

```
);
```

```
ALTER TABLE PokeDex
```

```
ADD FOREIGN KEY (tipo_id) REFERENCES Tipo(id);
```

```
ALTER TABLE PokeDex
```

```
ADD FOREIGN KEY (tipo_2_id) REFERENCES Tipo(id);
```

```
ALTER TABLE PokeDex
```

ADD FOREIGN KEY (move_id) REFERENCES Movimento(id);

ALTER TABLE PokeDex

ADD FOREIGN KEY (geracao_id) REFERENCES Geracao(id);

ALTER TABLE PokeDex

ADD FOREIGN KEY (raridade_id) REFERENCES Raridade(id);

ALTER TABLE Time_Pokemon

ADD FOREIGN KEY (treinador_id) REFERENCES Treinador(id);

ALTER TABLE Time_Pokemon_Selecao

ADD FOREIGN KEY (time_pokemon_id) REFERENCES Time_Pokemon(id);

ALTER TABLE Time_Pokemon_Selecao

ADD FOREIGN KEY (pokedex_id) REFERENCES PokeDex(id)

Consultas SQL associadas ao projeto (15 consultas)

1. Escolher por nome

```
SQL: SELECT p.id, p.nome FROM pokedex p
      WHERE
      p.nome = 'Groudon'
```

2. Escolher por id

```
SQL: SELECT p.id, p.nome FROM pokedex p
      WHERE
      p.id = 254
```

3. Escolher por raridade

```
SQL: SELECT p.id, p.nome FROM pokedex p
      JOIN raridade AS r
      ON p.raridade_id = r.id
      WHERE
      r.nome = 'Lendário'
```

4. Escolhe por dois filtros ex: geração e raridade

```
SQL: SELECT p.id, p.nome FROM pokedex p
      JOIN geracao AS g
      ON p.geracao_id = g.id
      JOIN raridade AS r
      ON p.raridade_id = r.id
      WHERE
      g.nome = 'Johto' AND r.nome = 'Pseudo-lendário'
```

5. Escolher o tipo independente de ser o primeiro ou o segundo

```
SQL: SELECT p.id, p.nome FROM pokedex p
      JOIN tipo AS t
      ON t.id IN (p.tipo_id, p.tipo2_id)
      WHERE
      t.nome = 'Poison'
```


6. Batalha

```
SQL: SELECT
    t1.posicao,
    p1.nome AS pokemon_time1,
    p2.nome AS pokemon_time2,
    m1.power AS power_time1,
    m2.power AS power_time2,
    CASE
        WHEN m1.power > m2.power THEN p1.nome
        WHEN m2.power > m1.power THEN p2.nome
        ELSE 'EMPATE'
    END AS vencedor
FROM time_pokemon_selecao t1
JOIN time_pokemon_selecao t2
    ON t1.posicao = t2.posicao
    AND t2.time_pokemon_id = 3 /* id do segundo treinador */
JOIN pokedex p1 ON p1.id = t1.pokedex_id
JOIN pokedex p2 ON p2.id = t2.pokedex_id
JOIN movimento m1 ON m1.id = p1.move_id
JOIN movimento m2 ON m2.id = p2.move_id
WHERE t1.time_pokemon_id = 1 /* id do primeiro treinador */
ORDER BY t1.posicao;
```

7. Mostrar quantas vitórias (para os céticos)

```
SQL: SELECT
    SUM(CASE WHEN m1.power > m2.power THEN 1 ELSE 0 END) AS
    vitorias_rodada_time1,
    SUM(CASE WHEN m2.power > m1.power THEN 1 ELSE 0 END) AS
    vitorias_rodada_time2
FROM time_pokemon_selecao t1
JOIN time_pokemon_selecao t2
    ON t1.posicao = t2.posicao
    AND t2.time_pokemon_id = 3 /* id do segundo treinador */
JOIN pokedex p1 ON p1.id = t1.pokedex_id
JOIN pokedex p2 ON p2.id = t2.pokedex_id
JOIN movimento m1 ON m1.id = p1.move_id
JOIN movimento m2 ON m2.id = p2.move_id
WHERE t1.time_pokemon_id = 4; /* id do primeiro treinador */
```

8. Mostrar os 6 pokémons de um time

```
SQL: SELECT
    p.id,
```

```

    p.nome
FROM time_pokemon tp
JOIN time_pokemon_selecao tps
    ON tps.time_pokemon_id = tp.id
JOIN pokedex p
    ON p.id = tps.pokedex_id
WHERE tp.id = 2
ORDER BY tps.posicao

```

9. Mostrar quais pokemons tem determinado movimento

```

SQL: SELECT
    p.id,
    p.nome AS pokemon,
    m.nome AS movimento,
    m.power
FROM pokedex p
JOIN movimento m
    ON m.id = p.move_id
WHERE m.nome = 'Pound'
ORDER BY p.nome;

```

10. Mostrar os pokémons de um treinador

```

SQL: SELECT DISTINCT
    tp.id      AS time_id,
    p.id,
    p.nome AS pokemon
FROM treinador tr
JOIN time_pokemon tp
    ON tp.treinador_id = tr.id
JOIN time_pokemon_selecao tps
    ON tps.time_pokemon_id = tp.id
JOIN pokedex p
    ON p.id = tps.pokedex_id
WHERE tr.id = 1
ORDER BY tp.id, p.nome;

```

11. Mostrar qual o pokémon mais forte

```

SQL: SELECT
    p.nome AS pokemon,
    m.nome AS movimento,
    m.power
FROM pokedex p

```

```
JOIN movimento m ON m.id = p.move_id
WHERE m.power = (
    SELECT MAX(power) FROM movimento
);
```

12. Média de poder de cada time de um treinador

```
SQL: SELECT
    tp.id AS time_id,
    AVG(m.power) AS media_poder
FROM treinador tr
JOIN time_pokemon tp ON tp.treinador_id = tr.id
JOIN time_pokemon_selecao tps ON tps.time_pokemon_id = tp.id
JOIN pokedex p ON p.id = tps.pokedex_id
JOIN movimento m ON m.id = p.move_id
WHERE tr.id = 1
GROUP BY tp.id;
```

13. Mostrar o menor ao maior pokemon de um time (power)

```
SQL: SELECT
    p.nome AS pokemon,
    m.nome AS movimento,
    m.power AS poder
FROM time_pokemon tp
JOIN time_pokemon_selecao tps
    ON tps.time_pokemon_id = tp.id
JOIN pokedex p
    ON p.id = tps.pokedex_id
JOIN movimento m
    ON m.id = p.move_id
WHERE tp.id = 1
ORDER BY m.power ASC;
```

14. Mostrar o top 5 movimento mais forte

```
SQL: SELECT
    nome,
    power
FROM movimento
ORDER BY power DESC
LIMIT 5;
```

15. Mostrar qual o pokémon mais utilizado de um treinador

```
SQL: SELECT
    p.nome AS pokemon,
    COUNT(*) AS vezes_usado
FROM treinador tr
JOIN time_pokemon tp ON tp.treinador_id = tr.id
JOIN time_pokemon_selecao tps ON tps.time_pokemon_id = tp.id
JOIN pokedex p ON p.id = tps.pokedex_id
WHERE tr.id = 1
GROUP BY p.nome
ORDER BY vezes_usado DESC
LIMIT 1;
```